Client Interface (CIF) Full Microarchitecture Specification

Contents

1 Overview

The Client Interface (CIF) accepts AXI read and write transactions, validates them, divides each accepted transaction into row-bounded MC packet segments, and returns AXI responses in order for each AXID. Read and write traffic use independent instances of the same transaction-tracking ROB architecture.

1.1 Global assumptions

- AXID is 6 bits (64 AXI IDs); each ROB instance has 64 per-AXID Slot FIFOs, each of depth four.
- One accepted AXI transaction produces 1–4 packet segments. AXI beats are represented by a segment's `beat_cnt`; a beat is not an MC packet.
- An AXI transaction must not cross 4 KB. With runtime-configured `row_size` of at least 1 KB, row crossings are at most three and `MAX_SEGMENTS=4`.
- MC Core echoes the packet's `GLOBAL_TAG` on completion. Completion-status encoding is **TBD with MC Core**.

2 Parameters

Parameter	Value
`AXID_W`	6
`ROB_DEPTH`	256 transactions per ROB instance
`ROB_INDEX_W`	8
`PACKET_NUMBER_W`	2
`GLOBAL_TAG_W`	10: {`ROB_INDEX, PACKET_NUMBER`}
`MAX_SEGMENTS`	4
Per-AXID Slot FIFO	64 FIFOs × depth 4, per ROB instance
Read Data Buffer watermark	128 entries, owned independently by that buffer

3 Transaction identity and packet formation

At AXI transaction acceptance, the applicable ROB Free List allocates `ROB_INDEX[7:0]`. The index is pushed into the accepting AXID's Slot FIFO only after the Burst Splitter has supplied `splitter_pkt_count`. The ROB initializes `exp_pkts=splitter_pkt_count` and clears its four-bit completion bitmap before the entry becomes eligible for retirement.

At packet emission, the Burst Splitter assigns `PACKET_NUMBER` from its two-bit `seg_idx` and Packet Formation attaches `GLOBAL_TAG={ROB_INDEX,PACKET_NUMBER}`. This is the complete MC completion identity.

4 Write path

4.1 Request path

The AW Metadata FIFO, W Data FIFO, Request Validator, Error Handler, Burst Splitter, Packet Formation Unit, and Write Packet FIFO retain their existing roles. The accepting write ROB

allocates the transaction index. The splitter generates one to four row-bounded segments and the Write Packet FIFO forwards one packet per segment to MC Core.

4.2 Write ROB and response

The write ROB has its own 256-entry Free List, Metadata RAM, and 64 Slot FIFOs. Its Metadata RAM is directly indexed by `ROB_INDEX` and stores `exp_pkts`, `completion_bitmap[3:0]`, and aggregate `resp_status`. MC completions set the bit selected by `PACKET_NUMBER` and update aggregate status. The status mapping and the worst-status precedence are **TBD**; no value is assumed here.

When the head transaction of an AXID Slot FIFO has all expected completion bits set, retirement selects it in AXID order. It holds BRESP valid until the downstream AXI response path accepts it. Only that acceptance pops the Slot FIFO and returns `ROB_INDEX` to the write Free List. Write traffic has no Read Data Buffer.

5 Read path

5.1 Request path

The AR Metadata FIFO, Request Validator, Error Handler, Burst Splitter, Packet Formation Unit, and Read Packet FIFO retain their existing roles. The read ROB allocates `ROB_INDEX` at acceptance. The splitter emits one to four segments, each carrying a distinct `PACKET_NUMBER`.

5.2 Read ROB, Data Buffer, and serializer

The independent read ROB has the same generic structures as the write ROB. Its Metadata RAM additionally stores `dbuf_ptr[0..3]`, one opaque Read Data Buffer allocation handle per packet number. At read packet issue, the handle for that segment's allocation is recorded before the packet is issued. Completion data is routed through that handle according to the MC/Data-Buffer interface contract, then `completion_bitmap[PACKET_NUMBER]` is set.

Once a Slot FIFO head is complete, a Retiring Register holds its `ROB_INDEX` while the Read Serializer emits the associated buffer allocations in packet-number order. The MC/Data-Buffer interface contract defines response-unit semantics, within-segment serialization, and allocation release. `RLAST` is asserted on the final AXI beat of the final segment; only after that beat and every associated contract-defined release may the Slot FIFO be popped and `ROB_INDEX` returned to the read Free List. Read Data Buffer handshakes, latency, pointer representation, and MC response-unit semantics remain **TBD**; its 128-entry watermark is its own capacity control, not ROB occupancy control.

6 Request and response packet formats

This is the compact, standalone reference for the CIF-generated request packet and the MC completion response, gathering fields already established in §3, §A, and §F. The request format (CIF→MC) and the response format (MC→CIF) carry different degrees of freeze and must not be conflated.

6.1 CIF → MC Core request packet

Packet Formation emits exactly one MC request packet per Segment Table entry (§A). The packet is transferred to MC Core using the `PKT_VALID/PKT_READY` handshake; that handshake is an interface signal pair, not a field carried inside the packet. Every request packet, read or write, carries:

Field	Width	Meaning
`GLOBAL_TAG`	10	{`ROB_INDEX[7:0]`,`PACKET_NUMBER[1:0]`}; this segment's completi
`addr`	32	Segment start address
`beat_cnt`	8	AXI beats represented by *this segment*, not the transaction's pack
packet type	inherited, unconfirmed width	Distinguishes Write/Read/Err_write/Err_read; set by the Request

Write request packets additionally carry write data for the beats represented by `beat_cnt`. The current documents establish only that a W Data FIFO supplies this data on the write path (§4); the per-segment write-data field width, byte-enable representation, and its exact relationship to `beat_cnt` are not fixed by any current CIF document and remain **TBD**.

Read request packets carry no data field. The interpretation of `addr` and `beat_cnt` at the MC request interface is governed by the CIF–MC interface contract.

For WRAP bursts, the CIF-side WRAP Beat Address Table remains responsible for generating the per-beat address sequence; the exact representation of that sequence at the MC request interface is part of the CIF–MC interface contract and is not fixed by this document.

Packet-type width/encoding is not restated by the post-refactor architecture documents; the field's existence and the values it must distinguish (Write, Read, Err_write, Err_read) are established, but its bit width should be treated as inherited-but-unconfirmed, in the same manner as `status` (see the Unresolved/TBD discussion referenced from §F).

6.2 MC Core → CIF response interface

The current architecture requires only that a completion be attributable to one segment and carry a status:

Field	Width	Meaning
`GLOBAL_TAG`	10	Echoed back unmodified; decodes directly to `ROB_INDEX` and `PACKET_NUMBER`
`status`	TBD	Raw MC completion status; encoding **TBD**
`data`	TBD	Read response payload, read segments only

The following remain **MC/Data-Buffer interface-contract dependent** and are intentionally not fixed by this specification:

- response transport and handshake naming — no `RESP_VALID/RESP_READY` signal pair is assumed;
- response payload width;
- response-unit granularity, i.e. whether one response unit corresponds to one AXI beat, one fixed-width data unit, or another unit;
- within-segment serialization semantics.

One segment's completion sets exactly one `completion_bitmap` bit regardless of that segment's `beat_cnt` — it is not one bit per AXI beat. A segment completing is therefore not the same event as `RLAST` on the AXI R channel, which additionally requires that segment's beats to have been serialized and accepted downstream. See the read ROB's completion and read-instance handling (§F) for how this response is consumed once its transport is defined by that contract.

7 Ordering, capacity, and reset

Slot FIFO order is transaction acceptance order for each AXID and therefore enforces AXI response ordering. Metadata RAM capacity is transaction capacity only. A full Slot FIFO blocks allocation for that AXID; an empty Free List blocks allocation globally. The latter means all 256 transaction slots are allocated, not merely that one Slot FIFO is full. Read request issuance is controlled separately by pooled Read Data Buffer availability.

Reset is active-low, asserted asynchronously, and deasserted synchronously; §12 gives the full initialization sequence. It clears both ROB Free Lists, Metadata RAM state, Slot FIFOs, and Retiring Registers. The Read Data Buffer is reset and drained according to its own specification. A soft reset is entered only after outstanding responses have been accepted and both ROB instances have no allocated Slot FIFO entries.

8 Error handling

Invalid requests enter the normal packet path with the appropriate error packet type. They receive ordinary `GLOBAL_TAG`s and complete through the corresponding ROB. Error status is preserved until ordered retirement; the external mapping of raw MC status to AXI `BRESP/RRESP` remains **TBD with MC Core**.

9 Top-level AXI4 interface

CIF presents a standard AXI4 slave interface to its client(s). The table below cross-references the channel signals against the widths the current architecture documents establish (`ADDR_W=32`, `AXID_W=6`, `LEN_W=8`, from the Request Validator's Design Parameters). Widths marked AXI4-fixed follow the AXI4 protocol specification itself and are not CIF-specific parameters. Write/read data-path widths are not established by any current CIF document and remain **TBD**; see §6.2 for the corresponding response-side discussion.

Channel	Signal	Width	Notes
AW	`AWVALID/AWREADY`	1/1	Write address handshake
	`AWADDR`	32	`ADDR_W`
	`AWID`	6	`AXID_W`
	`AWLEN`	8	`LEN_W`; *not* the MC packet/segment count (§3)
	`AWSIZE/AWBURST`	3/2	AXI4-fixed
W	`WVALID/WREADY`	1/1	Write data handshake
	`WDATA/WSTRB`	TBD	Data-path width not established by current CIF documents (§6)
	`WLAST`	1	AXI4-fixed

B	BVALID/BREADY	1/1	Write response handshake
	BID	6	AXID_W; from the retiring transaction's Slot FIFO context, not stored in the Metadata RAM entry
	BRESP	2	AXI4-fixed; driven from the write ROB's aggregate resp_status
AR	ARVALID/ARREADY	1/1	Read address handshake
	ARADDR	32	ADDR_W
	ARID	6	AXID_W
	ARLEN	8	LEN_W; *not* the MC packet/segment count (§3)
	ARSIZE/ARBURST	3/2	AXI4-fixed
R	RVALID/RREADY	1/1	Read data handshake
	RDATA	TBD	Data-path width not established by current CIF documents (§6)
	RID	6	AXID_W; from the retiring transaction's Slot FIFO context
	RRESP	2	AXI4-fixed; per-beat value from the read ROB's serialized status
	RLAST	1	Asserted only on the final AXI beat of the final segment, not on every segment boundary

Table 1: Top-level AXI4 slave interface, cross-referenced to CIF-internal handling

10 Interface Signal Table

This table records the consolidated interfaces after the read/write ROB refactor. Names describe the frozen connectivity where prior documents did not fix a signal name.

10.1 AXI request acceptance to the selected ROB

Signal	Width	Path	Description
alloc_req	1	Accept logic → RD/WR ROB	Allocate one transaction slot on accepted AR/AW request.
AXID	6	Accept logic → RD/WR ROB	Selects the per-AXID Slot FIFO.
ROB_INDEX	8	RD/WR ROB → Splitter	Free-List result retained for the transaction.
splitter_pkt_count	3	Splitter → RD/WR ROB	Number of segments for this transaction, 1–4; initializes exp_pkts.

10.2 Burst Splitter to Packet Formation

Signal	Width	Path	Description
seg_valid/seg_ready	1/1	Splitter ↔ Packet Formation	Handshake for the current Segment Table entry.

seg_addr	32	Splitter → Packet Formation	Segment start address.
seg_beat_cnt	8	Splitter → Packet Formation	Number of AXI beats represented by the segment.
PACKET_NUMBER	2	Splitter → Packet Formation	Current seg_idx, assigned at packet emission.
GLOBAL_TAG	10	Packet Formation → Packet FIFO	{ROB_INDEX,PACKET_NUMBER}.

10.3 MC Core completion interface

Response transport and handshake naming are MC/Data-Buffer interface-contract dependent (§6.2); no RESP_VALID/RESP_READY signal pair is assumed. The fields below are the only ones the current architecture requires MC Core to supply.

Signal	Width	Path	Description
GLOBAL_TAG	10	MC Core → RD/WR ROB	Echoed request tag; directly decodes index and packet number.
data	TBD	MC Core → Read ROB	Read data payload; width and response-unit granularity are contract-dependent (§6.2).
status	4	MC Core → RD/WR ROB	Raw completion status; encoding is **TBD with MC Core**.

10.4 Retirement interfaces

Signal	Width	Path	Description
merge_ready	1	Merge Logic → Read ROB	Accepts an emitted AXI read beat; serializer progress and Data Buffer release follow the MC/Data-Buffer contract.
rob_last	1	Read ROB → Merge Logic	Asserted with the final serialized read beat.
bresp_ready	1	B response path → Write ROB	Accepts the ordered BRESP and permits write-slot release.
dbuf_alloc/read/free	TBD	Read ROB ↔ Read Data Buffer	Allocation representation, handshake, response-unit semantics, and release timing are **TBD**; one handle is recorded per packet number.

11 Arbitration and backpressure

The request validators and Burst Splitter retain the existing request arbitration policy. The selected ROB Free List and selected AXID Slot FIFO provide transaction-allocation backpressure. A full Slot FIFO affects only its AXID; it does not imply global Free List exhaustion. Conversely, an empty Free List means all 256 transaction entries are allocated.

Read packet issuance is additionally controlled by pooled Read Data Buffer availability and its independent 128-entry watermark. Metadata RAM capacity does not control packet issue.

12 Reset and initialization sequence

Reset is active-low, asserted asynchronously, and deasserted synchronously to the CIF clock domain. While asserted, it forces every structure below to its initialized state; normal operation begins only after synchronous deassertion.

On reset, each ROB instance independently initializes its 256-bit Free List to free, clears Metadata RAM state, resets all 64 Slot FIFOs, and clears its Retiring Register. The Burst Splitter clears its Segment Table validity, `num_segments`, `seg_idx`, and retained `ROB_INDEX`. Packet FIFOs and AXI request FIFOs retain their established reset behavior. The MC completion/response handshake is not assigned a fixed reset value by this specification; its behavior across reset is governed by the MC–CIF interface contract (§6.2).

A soft reset first quiesces request acceptance and waits for packet FIFOs, Segment Table activity, both ROB Slot-FIFO sets, both Retiring Registers, and all externally visible AXI responses to drain. The Read Data Buffer has a separate drain/initialization contract; its exact interface and timing are **TBD**. Reset must not release a read `ROB_INDEX` until the final AXI read response and its Data Buffer release have completed.

13 Error propagation

Error packets are segmented and tagged in the same manner as ordinary requests. Every completion sets its packet-number bit in the selected ROB. The write ROB aggregates packet status before its one BRESP; the read ROB serializes status with its data. Raw MC status encoding and write worst-status precedence are **TBD**; this specification intentionally does not assign an AXI response value without that contract.

A Burst Splitter

A.1 Purpose and bounds

The Burst Splitter converts one accepted AXI transaction into one to four MC packet segments. It is shared by the read and write request paths. A row boundary is the only splitting boundary: AXI beats are carried by a segment's `beat_cnt`; they are not independently converted into MC packets.

The configured `row_size` is a run-time parameter and is at least 1 KB. AXI4 prohibits a burst from crossing a 4 KB boundary, so a transaction spans at most 4 KB. It can therefore cross at most three 1 KB rows and has at most four segments:

$$\texttt{MAX\_SEGMENTS} = 1 + 3 = 4.$$

A.2 Interfaces and state

At transaction acceptance, the selected read or write ROB allocates `ROB_INDEX[7:0]`. The splitter accepts that index with the AXI metadata and retains it for the whole transaction. Stage 1 determines `num_segments` and writes the Segment Table. It presents `splitter_pkt_count[2:0]=num_segments`

to the corresponding ROB; this is the sole source of `exp_pkts`. It is not derived from `AWLEN+1` or `ARLEN+1`.

State	Width/depth	Meaning
Segment Table	4 entries	{`addr[31:0], beat_cnt[7:0]`} per segment
`ROB_INDEX`	8 bits	Transaction index retained while segments issue
`num_segments`	3 bits	Number of valid Segment Table entries, 1–4
`seg_idx`	2 bits	Segment currently being emitted

A.3 Stage 1 — segment construction

Stage 1 receives address, burst length, size, burst type, AXID, packet type, and `ROB_INDEX`. Using the run-time `row_size`, it walks the addressed burst span and partitions it only at row boundaries. It writes one Segment Table entry for each contiguous portion of the transaction. The resulting count is in the inclusive range 1–4. The AXI 4 KB constraint and minimum row size make a fifth entry unreachable.

For a read allocation, Stage 1 also provides the initialized count to the read ROB before its `ROB_INDEX` becomes visible at the relevant per-AXID Slot FIFO. The write path follows the identical ordering.

A.4 Stage 2 — sequential emission

Stage 2 emits Segment Table entries in increasing `seg_idx` order. For each accepted packet it forms

`GLOBAL_TAG[9:0]={ROB_INDEX[7:0],PACKET_NUMBER[1:0]}`, `PACKET_NUMBER=seg_idx`.

Packet Formation receives `addr`, `beat_cnt`, packet type, and `GLOBAL_TAG`; it serializes one MC request packet per Segment Table entry. For reads, the packet-issue path also obtains a pooled Read Data Buffer pointer and records it in `dbuf_ptr[seg_idx]` before that packet can complete.

Stage 2 advances `seg_idx` only on downstream packet acceptance. It stalls Stage 1 whenever it is emitting a multi-segment transaction, thereby preserving the Segment Table and its associated `ROB_INDEX`. After the final accepted segment it returns to idle and permits the next Stage 1 transaction.

A.5 WRAP addressing within the Segment Table model

The Segment Table and WRAP address generation solve different problems and operate together for a WRAP burst that crosses a row boundary:

- Stage 1's row-boundary partitioning applies identically to INCR and WRAP bursts: it determines *how many* row-bounded segments the transaction requires and each segment's `addr`/`beat_cnt` starting point, independent of burst type.
- WRAP address generation determines the *per-beat* address sequence *within* a segment. Because a WRAP burst's beat addresses are not simply contiguous beyond its wrap boundary, the beat-by-beat address a WRAP segment presents to MC Core is produced by WRAP address generation, not by incrementing `addr` by the transfer size each beat.
- Both mechanisms are required together for a WRAP burst that crosses one or more row boundaries: the Segment Table determines segment count and each segment's starting point, while WRAP address generation determines the beat addresses used within each segment. Neither mechanism substitutes for the other.

This preserves the existing WRAP Beat Address Table semantics — a sequential, per-beat address sequence consumed in beat order — scoped now to one Segment Table entry at a time rather than to the whole burst.

A.6 Backpressure and error packets

- A full selected ROB Slot FIFO, an empty selected ROB Free List, or unavailable read Data Buffer capacity prevents acceptance upstream as appropriate to that path.
- Packet-FIFO backpressure holds the current Segment Table entry and its `seg_idx`; it does not alter `num_segments`.
- Error read and error write transactions use the same segmentation and `GLOBAL_TAG` mechanism. Their packet type is carried in every generated segment.

A.7 Timing notes

- `splitter_pkt_count` is the number of packets generated by this one transaction, not a sum across a burst.
- `seg_idx` is assigned at packet emission, not allocation.
- Segment Table state is indexed only by `seg_idx`; packet counting is independent of AXI beat count.

B Request Validator

B.1 Overview

The Request Validator is a purely combinational block instantiated once each on the read and write paths. It pops entries from the AR or AW Metadata FIFO, performs a single runtime check (address range validation), and steers the transaction to one of two downstream paths:

- **Valid path:** Transaction forwarded to ROB allocation with `REQ_VALID` asserted.
- **Error path:** Transaction forwarded to the Error Handler with `ERR_VALID` asserted and `AXID` preserved.

The Request Validator contains a Configuration Register that stores the valid physical address range used for runtime address validation. Apart from this internal configuration state, the Request Validator contains no per-transaction storage or counters. All protocol-level checks (burst type, WRAP legality, alignment, burst length) are enforced as design-time assertions in the verification environment only. Compliant AXI4 masters are assumed at runtime.

B.2 Design Parameters

Parameter	Value	Description
`ADDR_W`	32	Address width in bits
`AXID_W`	6	AXI ID width
`LEN_W`	8	Burst length field width (ARLEN/AWLEN)

Table 6: Request Validator parameters

B.3 Internal Blocks

Configuration Register Stores the valid physical address range used during request validation. Provides the configured address range to the Address Range Check block. Represents the only persistent state within the Request Validator.

B.4 Address Range Check

- **Type:** Purely combinational.
- **Input:** `ADDR[31:0]` from the AR/AW Metadata FIFO.
- **Function:** Checks whether `ADDR` falls within the valid physical address range stored in the Configuration Register.
- **Output:** `addr_ok` (1-bit) — asserted when the address is valid, deasserted on an address error.
- **Scope:** Address range check only. No burst type, no alignment, no length checks at runtime.

B.5 Output Mux

- **Type:** Purely combinational.
- **Function:** Routes the transaction to the correct downstream block based on `addr_ok`.
- **Valid path** (`addr_ok == 1`): asserts `REQ_VALID` to ROB allocation; forwards `AXID[5:0]`.
- **Error path** (`addr_ok == 0`): asserts `ERR_VALID` to the Error Handler; forwards `AXID[5:0]`.
- Only one output path is active per transaction. Both paths are never asserted simultaneously.

B.6 Interface Signals

B.7 Inputs

Signal	Width	Source	Description
`ADDR[31:0]`	32	AR/AW Metadata FIFO	Transaction address
`AXID[5:0]`	6	AR/AW Metadata FIFO	AXI ID
`ARLEN/AWLEN[7:0]`	8	AR/AW Metadata FIFO	Burst length
`STALL`	1	ROB Allocation	Per-AXID stall; holds output until deasserted

Table 7: Request Validator input signals

B.8 Outputs

Signal	Width	Destination	Description
`REQ_VALID`	1	ROB allocation	Valid transaction request
`AXID[5:0]`	6	ROB allocation	Forwarded AXI ID
`ERR_VALID`	1	Error Handler	Address error notification
`AXID[5:0]`	6	Error Handler	AXI ID preserved for error-response handling

Table 8: Request Validator output signals

B.9 Functional Behaviour

1. Request Validator pops the head entry from the AR/AW Metadata FIFO: `ADDR[31:0]`, `AXID[5:0]`, `ARLEN/AWLEN[7:0]`.
2. Address Range Check compares the incoming address against the physical address range stored in the Configuration Register and evaluates `addr_ok` combinationally.
3. If `addr_ok == 1` and `STALL` is deasserted: Output Mux asserts `REQ_VALID` to ROB allocation with `AXID`.
4. If `addr_ok == 0`: Output Mux asserts `ERR_VALID` to the Error Handler with `AXID`. The Error Handler sets `pkt_type = Err_write / Err_read` and injects the transaction into the normal pipeline via ROB allocation.
5. If `STALL` is asserted (ROB Allocation unavailable for this AXID): the Request Validator holds its outputs stable and does not pop the next FIFO entry until `STALL` deasserts.

B.10 Timing Notes

- Fully combinational. Output (`REQ_VALID` or `ERR_VALID`) is valid in the same cycle as the FIFO head entry is presented.
- `STALL` from ROB allocation is also combinational — the combined path (FIFO output → Address Range Check → Output Mux → ROB Allocation Stall Logic → STALL back) is a combinational loop that must be timed carefully. A register stage may be inserted between the Request Validator output and the ROB Allocation if timing closure requires it, at the cost of one cycle of allocation latency.
- The Address Range Check remains purely combinational. The Configuration Register provides the address range used during validation and is treated as stable during normal request processing.

B.11 Edge Cases

- **Error path stall:** An address error transaction still consumes a ROB allocation. If the AXID is at maximum outstanding (`STALL` asserted), the error path is also held. `ERR_VALID` is not asserted until `STALL` deasserts.
- **Protocol violations:** Illegal burst type, unaligned WRAP, and maximum burst length violations are not checked at runtime. They are enforced as design-time assertions in the verification environment. A non-compliant master will cause undefined behaviour.
- **Simultaneous valid and error:** Not possible — `addr_ok` is a single-bit result; exactly one output path is active per transaction.

C Transaction Allocation and ROB-Index Assignment

Transaction allocation is performed independently by the read and write ROB instances. The allocator's result is an eight-bit `ROB_INDEX`.

C.1 Structures and parameters

Structure	Definition
Free List	256-bit free/occupied bitmap per ROB instance
Allocation result	`ROB_INDEX[7:0]` selected from a free bit
Ordering storage	64 per-AXID Slot FIFOs × depth 4 per instance
Completion identity	`GLOBAL_TAG={ROB_INDEX,PACKET_NUMBER}`

C.2 Allocation flow

1. Request acceptance selects either the read or write ROB and presents `alloc_req` and `AXID`.
2. If that AXID's Slot FIFO is full, allocation for that AXID stalls. If its Free List is empty, allocation stalls globally for that ROB instance.
3. The Free List selects a free `ROB_INDEX` and reserves it.
4. Burst Splitter Stage 1 produces `splitter_pkt_count`; the Metadata RAM entry is initialized with that count and a zero completion bitmap.
5. Only after initialization is the index pushed to the AXID Slot FIFO and made visible to retirement. The index then accompanies the transaction to the Burst Splitter.

C.3 Release flow

Completion is not release. The write ROB returns an index only after the ordered BRESP is accepted. The read ROB returns an index only after its final AXI R response is fully accepted and the corresponding Read Data Buffer allocations have been released. Retirement performs the Slot FIFO pop and Free List update; no external free-by-tag interface exists.

C.4 Removed legacy behavior

The Slot FIFOs enforce the depth-four per-AXID outstanding limit and preserve AXI response order. A full Slot FIFO does not imply that the 256 entry Free List is empty; global exhaustion occurs only when all transaction capacity is allocated.

D Write Completion Tracking and BRESP Retirement

Write completion tracking is implemented inside the write ROB instance described in §F.

D.1 Write ROB metadata

Each write Metadata RAM entry is directly addressed by `ROB_INDEX` and contains:

- `exp_pkts[2:0]`, initialized from `splitter_pkt_count`;
- `completion_bitmap[3:0]`, initially zero; and
- `resp_status[1:0]`, initially benign pending the defined MC-to-AXI status mapping.

For each MC completion, the write ROB decodes `GLOBAL_TAG` directly, sets the bit selected by `PACKET_NUMBER`, and folds raw status into `resp_status`. The raw MC status encoding and the precedence rule when segment statuses differ are **TBD**. No counter derived from AXI burst length is used.

D.2 Ordered response and release

The head index of each per-AXID Slot FIFO is eligible when its completion bitmap equals the mask implied by `exp_pkts`. A priority selection loads an eligible index into the write Retiring Register. The write response logic presents one BRESP for that transaction and holds it until `bresp_ready`. Only on that acceptance does retirement pop the selected Slot FIFO and return `ROB_INDEX` to the write Free List.

This provides per-AXID AXI B ordering without coupling write release to read traffic.

E Read Completion Tracking and Serialization

Read completion tracking, final-beat indication, and release are implemented by the read ROB instance and its Retiring Register (§F).

E.1 Read ROB metadata and completion

The read Metadata RAM is directly indexed by `ROB_INDEX`. Each entry contains `exp_pkts[2:0]`, a four-bit completion bitmap, and one opaque Read Data Buffer allocation handle per `dbuf_ptr[0..3]`. The expected count is initialized from the Burst Splitter's `splitter_pkt_count`, which is the number of packet segments for this transaction.

On a read completion, `GLOBAL_TAG[9:0]` directly selects the entry and its packet-number bit. The completion data is written into the pooled Read Data Buffer allocation identified by the handle recorded for that packet number. Exact allocation representation, MC response-unit semantics, allocation/read/free handshakes, and latency remain **TBD**; the ROB assumes only that the handle is valid from packet issue until the contract-defined release point.

E.2 Read retirement

An AXID Slot FIFO head can retire only after all expected completion bits are set. The Retiring Register then holds that index while the Read Serializer serializes the completed Data Buffer allocations in packet-number order. Within-segment progress and allocation release are defined by the MC/Data-Buffer interface contract. `rob_last` is generated with the final AXI beat of the final segment. Final AXI R acceptance, after all contract-defined allocation releases, permits retirement to pop the Slot FIFO and return `ROB_INDEX` to the read Free List. Completion alone never frees the index.

F Read and Write Reorder Buffers

F.1 Overview

The ROB is a generic transaction-tracking core, instantiated independently for the read path and the write path. Each instance owns a complete, separate copy of every structure described in §H — its own 256-entry Free List, its own 256-entry Metadata RAM, and its own bank of 64 Per-AXID Slot FIFOs (depth 4 each). Nothing is shared between the two instances. They differ only in what a completed transaction produces: the read instance additionally holds a Data Buffer association and drives a Read Serializer; the write instance holds an aggregate response-status field and drives `BRESP` generation directly, since a write completion carries no payload.

Each transaction is identified end-to-end by a single GLOBAL_TAG, described in §G. ROB_INDEX is the Metadata RAM address end to end.

A transaction becoming *complete* (all packets accounted for in completion_bitmap) is not the same event as a transaction being *freed*. A completed transaction remains allocated while its ordered AXI response is being emitted and accepted, and (read only) while its Data Buffer allocations are still being drained. §H.5 makes this lifecycle explicit.

Lifecycle Summary

The full life of a ROB entry, in order, referenced to the sections that detail each stage:

1. AXI request accepted (upstream of this file) — §M
2. ROB_INDEX allocated from the Free List — §H.1, §M
3. Metadata RAM initialized (exp_pkts, completion_bitmap = 0) — §M
4. Entry becomes visible in its AXID's Slot FIFO — §H.3, §M
5. Packets emitted, GLOBAL_TAG attached, (read) dbuf_ptr recorded — §N
6. Completions decoded, completion_bitmap updated — §H.4, §O
7. Transaction reaches completion; becomes eligible to retire — §H.5
8. Ordered AXI response emitted and accepted (read: beat by beat; write: single BRESP) — §I, §J
9. Slot FIFO popped, ROB_INDEX freed, (read) Data Buffer allocations already released — §H.5, §Q

Architecture principles

- Completion directly supplies ROB_INDEX and PACKET_NUMBER; RAM access is a bit-slice decode.
- Per-AXID ordering is carried entirely by the Per-AXID Slot FIFOs (§H.3).
- Packet position is the low 2 bits of GLOBAL_TAG (PACKET_NUMBER).
- Read request-issue backpressure is a function of the pooled Read Data Buffer's own 128-entry watermark, a separate structure outside this file's scope (see §R). No watermark applies to Metadata RAM.

G Global Tag

Field	Width
ROB_INDEX	8 bits (addresses one of 256 Metadata RAM entries)
PACKET_NUMBER	2 bits (segment index, 0..3)
GLOBAL_TAG	10 bits, {ROB_INDEX[7:0], PACKET_NUMBER[1:0]}

Generation. At allocation, the Allocator (§H.1) issues ROB_INDEX. At packet emission, the Burst Splitter supplies PACKET_NUMBER (seg_idx) for each segment of the transaction. Packet Formation concatenates the two into GLOBAL_TAG, which is attached to every packet sent to MC Core.

Carry. MC Core is not required to interpret GLOBAL_TAG — it echoes the value back unmodified on completion. The exact MC response-unit and burst semantics are defined by the MC–CIF interface contract and remain TBD.

Decode. On completion, Completion Input (§H.4) splits the returned `GLOBAL_TAG` directly: bits `[9:2]` address Metadata RAM (`ROB_INDEX`); bits `[1:0]` select the packet position within that entry (`PACKET_NUMBER`). Both are pure bit-slices — no lookup, no associative match.

H Generic Architecture

The blocks in this section are instantiated once per ROB instance (i.e. once for read, once for write), with identical logic and identical sizing in both — see Overview.

H.1 Allocator / Free List

- 256-bit free/occupied bitmask, one bit per `ROB_INDEX`. `1 = free`, `0 = occupied`.
- Allocate: priority encoder selects lowest free bit, clears it, and outputs `ROB_INDEX[7:0]`.
- Free: Retirement Logic sets the bit at `ROB_INDEX` via `free_req`, only once a slot has fully retired (§H.5) — never merely on completion.
- `FREE_LIST_EMPTY` (all bits 0) is a raw pool-wide allocation-backpressure signal. With `ROB_DEPTH = 256` and 64 × depth-4 Slot FIFO capacity, it means every transaction slot is allocated and all FIFO capacity is collectively exhausted. A single full Slot FIFO does not imply this condition. See §S.

H.2 Metadata RAM

- 256-entry RAM, directly addressed by `ROB_INDEX` (no CAM, no translation).
- Depth is fixed at `ROB_DEPTH = 256`; no watermark or occupancy threshold applies to this RAM (see Overview). Allocation is gated by Free List exhaustion or per-AXID Slot FIFO fullness (§R); these are distinct conditions.
- Entry contents are defined explicitly in §L.
- Write port A (allocation): Allocator writes `exp_pkts` and clears `completion_bitmap` at `ROB_INDEX` on allocation, before the entry is exposed to retirement — see §M for the exact ordering.
- Write port B (completion): Completion Input updates `completion_bitmap` (both instances) and the instance-specific field (`dbuf_ptr` entry for read, `resp_status` for write) at `ROB_INDEX`.
- Read port: Retirement Logic reads the entry at the head-of-FIFO `ROB_INDEX` for each AXID to evaluate completion.
- No explicit `valid` bit. A Metadata RAM entry is only meaningful while its `ROB_INDEX` is occupied (cleared in the Free List) and present in a Per-AXID Slot FIFO; both conditions are tracked externally to the RAM.

H.3 Per-AXID Slot FIFO

- 64 independent FIFOs, one per AXID, depth 4 each — frozen sizing, identical for both instances. Each entry stores `ROB_INDEX[7:0]`.
- Push: only after the corresponding Metadata RAM entry has been initialized (`exp_pkts`, `completion_bitmap = 0`) — see §M. An entry is never visible in a Slot FIFO before it is safe to read.
- Pop: not on completion. Only when the transaction at the head has fully retired — i.e. its ordered AXI response has been emitted and accepted (§H.5).

- FIFO order = allocation order = required AXI response order for that AXID, by construction. No separate ordering structure is needed.
- FIFO-full for a given AXID is the per-AXID backpressure condition on allocation. Because a completed-but-not-yet-retired entry remains at the head until fully drained, the head slot may stay occupied for several cycles after completion during response emission — this is expected, not a fault condition.

H.4 Completion Input

- Receives the MC completion/response unit together with `GLOBAL_TAG` and `status`. The exact response transport, payload width, and handshake are defined by the MC–CIF interface contract.
- Decodes `GLOBAL_TAG` into `ROB_INDEX` and `PACKET_NUMBER` per §G.
- Sets `completion_bitmap[PACKET_NUMBER] = 1` at `ROB_INDEX`.
- Read instance: routes completion data and status to the Read Data Buffer allocation identified by `dbuf_ptr[PACKET_NUMBER]` (the handle was recorded at packet-emission time, not derived here). The MC completion response-unit semantics and the Data Buffer write protocol are defined by the MC/Data-Buffer interface contract — see §I.
- Write instance: folds `status` into `resp_status` (worst-of; exact precedence: **TBD**, see §T) — see §J.
- Completion acceptance follows the MC–CIF response interface contract. The ROB does not impose a separate completion-side handshake beyond that contract.
- Completion Input only ever marks *completion*. It never pops a Slot FIFO and never frees a `ROB_INDEX` — both are the exclusive responsibility of Retirement Logic (§H.5).

H.5 Retirement Logic

Completion and retirement are distinct events. Completion (`completion_bitmap` full) makes a transaction *eligible* to retire; it does not by itself release anything. Release happens only after the transaction's ordered AXI response has been fully accepted.

- **Retiring Register** (generic, minimal addition): a single register per instance holding the `ROB_INDEX` currently being drained, plus read-instance progress state consisting of a `PACKET_NUMBER` segment pointer and any within-segment progress state required by the MC/Data-Buffer interface contract. The write instance requires no analogous data-stream progress state. At most one transaction is draining at a time per instance, consistent with the single response-channel path each instance already serves.
- Each cycle, while the Retiring Register is idle, Retirement Logic reads the Metadata RAM entry at the head `ROB_INDEX` of every AXID's Slot FIFO and checks `completion_bitmap == (1 << exp_pkts) - 1`.
- When idle, a priority encoder selects the lowest-numbered AXID whose head entry is complete and loads its `ROB_INDEX` into the Retiring Register, entering the *retiring* state. **The Slot FIFO is not popped and `ROB_INDEX` is not freed at this point.**
- While retiring, instance-specific emission proceeds (§I for read, §J for write) until the transaction's full AXI response has been accepted downstream (`merge_ready` for read, `bresp_ready` for write — §K).
- Only once the full response has been accepted: Retirement Logic pops that AXID's Slot FIFO entry, returns `ROB_INDEX` to the Free List (`free_req`), and the Retiring Register returns to idle.

- Maximum one transaction draining at a time per instance; draining a single read transaction may itself span multiple cycles (a number of AXI beats determined by the segment `beat_cnt` values), unlike the previous single-cycle-emit assumption.

I Read-Instance Extension

State and logic in this section exist only in the read-path ROB instance.

I.1 Metadata RAM Extension

- `dbuf_ptr[0..3][7:0]`: one opaque allocation handle per possible `PACKET_NUMBER`, into the pooled Read Data Buffer. Written by the packet-emission path at issue time, one handle per segment — *not* written by Completion Input, which consumes the handle to route the response.
- A handle does not imply one physical buffer entry or one AXI beat. It identifies the Data Buffer allocation associated with that segment; whether that allocation is a region, a descriptor, or another structure is defined by the MC/Data-Buffer interface contract.

I.2 Read Data Buffer (cross-reference)

- The pooled Read Data Buffer, its independently managed 128-entry watermark, and its occupancy/backpressure logic are **out of scope for this file**. It is a separate structure the read ROB addresses through `dbuf_ptr`, not a substructure of Metadata RAM. Data-buffer capacity and watermark are independent of `ROB_DEPTH`/Metadata RAM capacity.
- Exact Data Buffer interface signal names, allocation representation, response-unit semantics, and timing are **TBD** in the MC/Data-Buffer interface contract — see §T.

I.3 Read Serializer

- Operates only while the Retiring Register (§H.5) holds a read transaction.
- Serializes the completed data represented by `dbuf_ptr[PACKET_NUMBER]` in segment order, tagged with the AXID selected from the Slot FIFO context. The number of AXI beats represented by a segment and the cursor/handshake needed to advance within that segment are defined by the MC/Data-Buffer interface contract; the ROB does not assume one response unit or one buffer entry per segment.
- `merge_ready` accepts the AXI beat currently presented by that contract. Data Buffer release occurs at the allocation granularity specified by that contract, after all data represented by the allocation has been accepted; it is not defined as one release per accepted AXI beat.
- `rob_last` is asserted on the final AXI beat of the final segment. Final-segment selection comes from `PACKET_NUMBER == exp_pkts - 1`; final-beat detection within that segment comes from the MC/Data-Buffer interface contract.
- Once the final AXI beat is accepted and the final segment's Data Buffer allocation has been released according to that contract, Retirement Logic pops the Slot FIFO and frees `ROB_INDEX` (§H.5).

Read-Instance I/O (in addition to generic I/O in §K)

Signal	Width	Dir / Peer	Description
data	TBD	in, MC Core	Read response data, according to the MC–CIF interface contract
data	TBD	out, Merge Logic	In-order read response data unit
status[3:0]	4	out, Merge Logic	Status for emitted beat
rob_last	1	out, Merge Logic	Asserted on the final AXI beat of the final segment
merge_ready	1	in, Merge Logic	Accept for the AXI beat currently emitted by the Read Serializer

Table 9: Read-instance additional signals

J Write-Instance Extension

State and logic in this section exist only in the write-path ROB instance.

J.1 Metadata RAM Extension

- resp_status[1:0]: aggregate BRESP value for the transaction. Initialized to OKAY at allocation; updated by Completion Input as each packet's status arrives, worst-of semantics (exact priority ordering among AXI response codes: **TBD**, see §T).

J.2 BRESP Generation

- Operates only while the Retiring Register (§H.5) holds a write transaction.
- Emits a single bresp = resp_status to the Response Dispatcher, tagged with the AXID currently being serviced, and holds it until accepted downstream (bresp_ready, §K).
- No data movement, no per-packet serialization — one response per transaction regardless of exp_pkts.
- Once bresp_ready accepts the response, Retirement Logic pops the Slot FIFO and frees ROB_INDEX (§H.5) — not before.

Write-Instance I/O (in addition to generic I/O in §K)

Signal	Width	Dir / Peer	Description
bresp[1:0]	2	out, Response Dispatcher	Aggregate write response for the transaction
bresp_ready	1	in, Response Dispatcher	Accept for bresp; gates Slot FIFO pop and ROB_INDEX free in Retirement Logic

Table 10: Write-instance additional signals

K Generic I/O

Signal	Width	Dir / Peer	Description

alloc_req	1	in, Accept/Request logic	Pulse: allocate a new ROB slot
AXID[5:0]	6	in, Accept/Request logic	AXID of the transaction being allocated; used only to select the Per-AXID Slot FIFO, not stored in Metadata RAM
exp_pkts[2:0]	3	in, Burst Splitter	splitter_pkt_count for this transaction
ROB_INDEX[7:0]	8	out, Accept/Request logic	Allocated slot index, returned for use in GLOBAL_TAG generation
MC_RESP	TBD	in, MC Core	MC completion/response unit; exact transport defined by the MC–CIF interface contract
GLOBAL_TAG[9:0]	10	in, MC Core	{ROB_INDEX[7:0], PACKET_NUMBER[1:0]}, echoed unmodified
status[3:0]	4	in, MC Core	Raw completion status/error code (mapping to AXI RESP encoding: **TBD**, see §T)

Table 11: Generic signals, present on both instances

Instance-specific downstream accept signals (merge_ready, bresp_ready) are listed with their respective instance's I/O table above, since only one instance's Retirement Logic consumes each.

L Metadata RAM Entry Definition

Field	Width	Scope
exp_pkts[2:0]	3 bits	generic
completion_bitmap[3:0]	4 bits	generic
dbuf_ptr[0..3][7:0]	$4 \times 8 = 32$ bits	read-only
resp_status[1:0]	2 bits	write-only

Total entry width: 39 bits (read instance), 9 bits (write instance). AXID is available from Slot FIFO context at retirement and is not stored in the entry. No "retiring" state is stored per entry either — that state lives entirely in the single Retiring Register described in §H.5, since at most one transaction per instance is ever draining at a time.

M Allocation Path

1. Accept/Request logic sends alloc_req with AXID, following AXI request acceptance upstream of this file.
2. Allocator pops ROB_INDEX from the Free List (priority encoder on the bitmask), clears the bit. ROB_INDEX is now reserved but not yet visible to Retirement Logic.
3. Once Burst Splitter Stage 1 computes exp_pkts (= splitter_pkt_count) for the burst, Metadata RAM is written at ROB_INDEX: exp_pkts set, completion_bitmap cleared to 0, and

(write instance) `resp_status` initialized to `OKAY`.

4. Only *after* this Metadata RAM initialization write completes is `ROB_INDEX` pushed onto the requesting AXID's Slot FIFO. This ordering is deliberate: the Slot FIFO push and the Metadata RAM initialization write are both gated on the same Burst Splitter Stage 1 completion, so an entry can never become visible to Retirement Logic with a stale or uninitialized `exp_pkts/completion_bitmap`.

N Packet Issue Path

1. For each segment `PACKET_NUMBER = 0..num_segments-1` emitted by the Burst Splitter:
2. (read instance) a Read Data Buffer allocation is obtained from its own pool; its opaque handle is written to `dbuf_ptr[PACKET_NUMBER]` in the Metadata RAM entry at `ROB_INDEX`.
3. Packet Formation attaches `GLOBAL_TAG = {ROB_INDEX, PACKET_NUMBER}` to the packet.
4. Packet is issued to MC Core via the Packet FIFO.

O Completion Path

1. The MC response unit arrives with `GLOBAL_TAG`, `status`, and, for reads, response data according to the MC–CIF interface contract.
2. Completion Input decodes `GLOBAL_TAG` into `ROB_INDEX` and `PACKET_NUMBER`.
3. (read instance) completion data is routed to the Read Data Buffer allocation identified by `dbuf_ptr[PACKET_NUMBER]`, according to the MC/Data-Buffer interface contract.
4. (write instance) `resp_status` is updated with the worst-of comparison against the incoming `status`.
5. `completion_bitmap[PACKET_NUMBER]` is set at `ROB_INDEX`.
6. Completion acceptance only marks the corresponding packet complete; it never triggers a Slot FIFO pop or a Free List return. Retirement remains responsible for releasing the ROB entry.

P Retirement Path

1. Each cycle, while the Retiring Register is idle, Retirement Logic reads the Metadata RAM entry at every AXID's Slot FIFO head.
2. Priority encoder selects the lowest AXID whose head entry satisfies `completion_bitmap == (1 << exp_pkts) - 1`; its `ROB_INDEX` is loaded into the Retiring Register. **The Slot FIFO entry is not popped yet.**
3. (read instance) Read Serializer drains the completed allocations represented by `dbuf_ptr[0..num_segments-1]` in segment order. Its within-segment progress and Data Buffer release behavior follow the MC/Data-Buffer interface contract.
4. (write instance) BRESP Generation emits `bresp` and holds it until `bresp_ready` accepts it.
5. Only once the transaction's full AXI response has been accepted (final read beat, or the single write response) does Retirement Logic pop that AXID's Slot FIFO and return `ROB_INDEX` to the Free List. The Retiring Register returns to idle.

Q Slot Release

1. Triggered only by full AXI response acceptance (§H.5) — never by `completion_bitmap` reaching its full mask on its own.
2. `ROB_INDEX` is returned to the Free List (`free_req`), bit set back to `1 = free`.
3. (read instance) each `dbuf_ptr[i]` allocation is returned to the Read Data Buffer's own pool at the allocation granularity and release point defined by the MC/Data-Buffer interface contract. The final transaction release requires all associated allocations to be released.
4. Retirement needs only the Slot FIFO pop and Free List return.

R Backpressure

Signal	Description
`Slot FIFO full (per AXID)`	Blocks new allocation for that AXID. A completed-but-still-draining head entry keeps the FIFO occupied until fully retired, so this condition can now persist slightly longer than "completion" alone would suggest — expected behavior, not a fault.
`FREE_LIST_EMPTY`	Blocks new allocation pool-wide. It means all 256 transaction slots are allocated and all Slot FIFO capacity is collectively consumed. A single full AXID FIFO does not imply Free List exhaustion.

`ROB_DEPTH = 256` carries no watermark of its own. Read-path issuance backpressure is driven entirely by the pooled Read Data Buffer's 128-entry watermark, a separate structure not detailed in this file.

S Key Parameters

Parameter	Value
Metadata RAM depth (`ROB_DEPTH`)	256 entries (`ROB_INDEX` is 8 bits) — frozen, identical for both instances
`PACKET_NUMBER` width	2 bits (`MAX_SEGMENTS = 4`) — frozen
`GLOBAL_TAG` width	10 bits — frozen
Entry width, read instance	39 bits
Entry width, write instance	9 bits
Free list width	256 bits, one independent Free List per instance
Per-AXID Slot FIFO	64 FIFOs × depth 4 — frozen, identical for both instances
Retiring Register	1 per instance: `ROB_INDEX[7:0]` + read progress state
Max transactions draining	1 per instance at a time; a single read transaction's drain may span multi

$64 \times 4 = 256$, exactly matching `ROB_DEPTH`. Thus `FREE_LIST_EMPTY` means all per-AXID FIFO capacities are collectively occupied; it remains distinct from the condition that any one FIFO is full.

T Unresolved / TBD

Every item below requires an external decision or a separate component's specification — none can be derived from the frozen ROB architecture itself.

- **MC Core raw status encoding.** The exact bit encoding MC Core uses on `status[3:0]` for packet-level completion status, and therefore the combinational mapping from `status[3:0]` to AXI `RESP[1:0]` used when updating `resp_status`, is not defined in any document available to this revision. Requires MC Core interface sign-off; `status[3:0]`'s width itself is inherited from the pre-refactor design and has not been reconfirmed against MC Core's actual encoding.
- **`resp_status` worst-of precedence.** When packets of the same write transaction return different status values (e.g. one `SLVERR`, one `DECERR`), the exact precedence used to compute the single aggregate `BRESP` is an internal policy decision for Completion Input, not fixed by the AXI4 specification or by any frozen decision here. Needs an explicit precedence rule before RTL can be written.
- **Pooled Read Data Buffer / MC response contract.** Signal names, allocation representation, response-unit semantics, and timing for allocating, filling, serializing, and freeing a Data Buffer allocation are defined by a separate MC/Data-Buffer interface specification. This file assumes only that `dbuf_ptr[PACKET_NUMBER]` is a valid segment allocation handle at packet-emission time and remains valid until the contract-defined release point.

U Timing Notes

- Metadata RAM read/write ports: allocation write, completion write, and retirement read are independent accesses; simultaneous allocation and completion in the same cycle to different `ROB_INDEX` values is safe, unchanged in spirit from the prior BRAM port arrangement.
- Priority encoder across 64 Slot FIFO heads in Retirement Logic remains the likely critical path; may need pipelining at high frequencies, as in the prior design. It only runs while the Retiring Register is idle.
- Simultaneous `alloc_req` and `free_req` on the Free List bitmask in the same cycle is safe — they target different bits by construction (allocate clears the lowest free bit; free sets the bit of the just-retired, and therefore currently occupied, slot).
- `GLOBAL_TAG` decode (§G) is a pure bit-slice and has no translation latency on the completion path.
- The Slot FIFO push at allocation is deliberately deferred until the Metadata RAM initialization write completes (§M) — this introduces a small, bounded latency between `alloc_req` and Slot FIFO visibility, bounded by Burst Splitter Stage 1's latency to produce `exp_pkts`.
- Draining a read transaction is explicitly multi-cycle through the Retiring Register. Transfer granularity and within-segment progress follow the MC/Data-Buffer interface contract, so Retirement Logic and the Read Serializer must be read as a small state machine (idle / retiring), not a single combinational selection. The write instance is analogous but single-beat, gated by `bresp_ready`.